

Projeto de Avaliação: Unidade 1

Java, Maven, Spring Boot e Testes

- Formato: individual ou grupo de até 7 participantes
- Prazo: 07/10/2026
- Entrega: repositório público no GitHub + apresentação curta
- Unidade: 1

1. Contexto

Durante a primeira unidade, trabalhamos a construção de aplicações Back-End utilizando Java e evoluímos progressivamente de uma aplicação simples para uma estrutura organizada em camadas.

Ao longo das aulas, trabalhamos:

- fundamentos de Java aplicados ao Back-End;
- orientação a objetos;
- classes, objetos, métodos e encapsulamento;
- interfaces;
- coleções;
- organização em packages;
- padrão Repository;
- gerenciamento de dependências com Maven;
- Spring Boot;
- Controllers;
- Services;
- injeção de dependências;
- organização de responsabilidades;
- testes automatizados com JUnit.

Este projeto é a avaliação prática da Unidade 1. O objetivo é demonstrar que vocês conseguem utilizar os conceitos estudados para construir uma aplicação Back-End pequena, organizada e testável; não é construir uma aplicação comercial completa.

2. Desafio

Cada aluno individualmente ou cada grupo deverá desenvolver uma aplicação Back-End em Java, utilizando Spring Boot, capaz de resolver um problema de negócio simples.

O sistema deverá possuir:

1. entidades de domínio;
2. regras de negócio;
3. organização em camadas;
4. persistência em memória;
5. gerenciamento de dependências com Maven;
6. utilização do Spring Boot;
7. Controllers;
8. Services;
9. Repositories;
10. injeção de dependências;
11. testes automatizados com JUnit;
12. documentação completa no README;
13. histórico de desenvolvimento no Git/GitHub.

3. Temas disponíveis

Escolham **um** dos temas abaixo.

Tema 1: Sistema de Biblioteca

Sistema para gerenciamento de livros, leitores e empréstimos.

Entidades sugeridas

- Livro
- Leitor
- Empréstimo

Funcionalidades mínimas

- cadastrar livro;
- cadastrar leitor;
- emprestar livro;
- devolver livro;
- listar livros;
- listar empréstimos;
- listar livros disponíveis;
- consultar pendências de um leitor.

Exemplos de regras de negócio

- livro indisponível não pode ser emprestado.

Tema 2: Sistema de Estacionamento

Sistema para controle de vagas, entradas e saídas de um estacionamento.

Entidades sugeridas

- Vaga
- Veículo
- Permanência

Funcionalidades mínimas

- cadastrar vaga;
- cadastrar veículo;
- registrar entrada;
- registrar saída;
- consultar valor a pagar;
- listar vagas livres;
- listar permanências.

Exemplos de regras de negócio

- vaga ocupada não pode receber outro veículo.

Tema 3: Sistema de Matrículas em Turmas

Sistema para gerenciamento de alunos, turmas e matrículas.

Entidades sugeridas

- Aluno
- Turma
- Matrícula

Funcionalidades mínimas

- cadastrar aluno;
- criar turma;
- matricular aluno em turma;
- cancelar matrícula;
- listar alunos;
- listar turmas;
- listar alunos de uma turma.

Exemplos de regras de negócio

- aluno deve existir antes da matrícula.

Tema 4: Sistema de Controle de Estoque

Sistema para gerenciamento de produtos e movimentações de estoque.

Entidades sugeridas

- Produto
- Categoria
- Movimentação

Funcionalidades mínimas

- cadastrar produto;
- cadastrar categoria;
- registrar entrada de estoque;
- registrar saída de estoque;
- consultar saldo de um produto;
- listar produtos;
- listar produtos abaixo do estoque mínimo.

Exemplo de regras de negócio

- produto deve existir antes da movimentação.

Tema 5: Sistema de Fila de Atendimento

Sistema para gerenciamento de senhas e atendimentos em um balcão.

Entidades sugeridas

- Senha
- Atendimento
- Atendente

Funcionalidades mínimas

- emitir senha;
- chamar próxima senha;
- iniciar atendimento;
- encerrar atendimento;
- listar senhas na fila;

- listar atendimentos;
- listar atendimentos por atendente.

Exemplos de regras de negócio

- senha não pode ser chamada duas vezes.

Tema 6: Tema próprio (mediante aprovação do professor)

Vocês podem propor um tema próprio. O problema precisa ser simples e compatível com uma semana de desenvolvimento, com aproximadamente 3 a 5 entidades principais e regras de negócio claras e verificáveis. A aprovação do professor deve vir antes do início do desenvolvimento: descrevam o tema em até 1 página, com o problema, as entidades sugeridas e algumas regras de negócio.

Os exemplos de código deste documento usam o cenário de aluguel de bicicletas, que foi trabalhado em aula e não está entre os temas. Eles servem para mostrar como uma regra se transforma em código e em teste; vocês devem fazer o mesmo com o tema que escolherem.

4. Nível de complexidade

O projeto deve ser compatível com uma semana de desenvolvimento e com o conteúdo trabalhado na Unidade 1. Não é necessário criar uma aplicação grande.

Um bom projeto terá:

- aproximadamente 3 a 5 entidades principais;
- regras de negócio claras;
- Services com responsabilidades bem definidas;
- Repositories em memória;
- Controllers;
- testes das principais regras;
- código organizado.

Não será necessário implementar banco de dados.

5. Tecnologias obrigatórias

O projeto deverá utilizar:

- Java 21
- Maven
- Spring Boot
- JUnit
- Git

- GitHub

A persistência deverá ser feita em memória, utilizando estruturas como `List / ArrayList` .

6. Tecnologias não recomendadas nesta avaliação

Os itens abaixo não são necessários e não devem substituir os conceitos trabalhados:

- PostgreSQL;
- MySQL;
- MongoDB;
- JPA;
- Hibernate;
- JDBC;
- Docker;
- APIs externas;
- autenticação;
- JWT;
- front-end;
- React;
- Vue;
- frameworks adicionais para substituir a implementação das regras.

O foco desta avaliação é demonstrar o domínio dos conteúdos trabalhados na U1.

7. Arquitetura obrigatória

A aplicação deverá seguir uma separação de responsabilidades semelhante a:

```
src/
└─ main/
   └─ java/
      └─ com/
         └─ exemplo/
            └─ projeto/
               │
               ├── Main.java
               │
               ├── controller/
               │   ├── ...
               │   └── ...
               │
               ├── model/
               │   ├── ...
               │   └── ...
               │
               ├── repository/
               │   ├── ...
               │   └── ...
               │
               └── service/
                   ├── ...
                   └── ...
```

A separação deve representar responsabilidades reais.

Controller

Responsável pela interação com a aplicação e pela entrada/saída de dados.

Service

Responsável pelas regras de negócio.

Repository

Responsável pelo armazenamento e recuperação dos dados.

Model

Representa as entidades do domínio.

Main

Responsável pela inicialização da aplicação Spring Boot.

8. Injeção de dependências

As dependências entre as camadas deverão ser fornecidas pelo Spring.

Por exemplo:

```
@Service
public class ExemploService {

    private final ExemploRepository repository;

    public ExemploService(
        ExemploRepository repository
    ) {
        this.repository = repository;
    }
}
```

A utilização de `new` para criar manualmente Services e Repositories no fluxo principal da aplicação deverá ser evitada.

O objetivo é demonstrar o conceito de injeção de dependência trabalhado em aula.

9. Repository em memória

O Repository deverá abstrair o armazenamento.

Exemplo:

```
public interface ExemploRepository {

    void salvar(Exemplo exemplo);

    List<Exemplo> listar();

    Exemplo buscarPorId(Long id);
}
```

Uma implementação em memória poderá utilizar:

```
@Repository
public class ExemploRepositoryMemoria
    implements ExemploRepository {

    private final List<Exemplo> exemplos =
        new ArrayList<>();
}
```

O objetivo é demonstrar a separação entre a regra de negócio e a forma como os dados são armazenados.

10. Regras de negócio

O projeto deverá possuir pelo menos 5 regras de negócio claramente identificáveis.

As regras precisam estar implementadas no local adequado, preferencialmente nos Services ou nas próprias entidades quando fizer sentido. Não basta escrever as regras no README: elas precisam existir no código e ser verificáveis.

Exemplo

Uma regra:

Uma bicicleta já alugada não pode ser alugada novamente.

Deverá resultar em comportamento equivalente a:

```
if (!bicicleta.estaDisponivel()) {  
    throw new IllegalStateException(  
        "A bicicleta já está alugada."  
    );  
}
```

11. Testes automatizados

O projeto deverá possuir testes utilizando JUnit.

Cada grupo deverá implementar pelo menos 5 testes automatizados relevantes.

Os testes devem verificar comportamentos do sistema, e não apenas executar métodos sem verificar resultados.

É obrigatório que exista pelo menos:

- 1 teste de comportamento válido;
- 2 testes de regras de negócio;
- 1 teste de situação inválida/exceção;
- 1 teste adicional escolhido pelo grupo.

Exemplo

```
@Test
void naoDevePermitirAlugarBicicletaJaAlugada() {

    bicicleta.alugar();

    assertThrows(
        IllegalStateException.class,
        bicicleta::alugar
    );
}
```

Os testes deverão ser executáveis através do Maven:

```
mvn test
```

O README deverá informar como executar os testes.

12. Maven e dependências

O projeto deverá possuir um `pom.xml` corretamente configurado.

As dependências utilizadas deverão ser declaradas pelo Maven. Não entregue bibliotecas `.jar` manualmente dentro do projeto para substituir o gerenciamento de dependências.

O README deverá informar as principais dependências utilizadas e sua finalidade.

13. Git e GitHub: requisito obrigatório

A entrega deverá ser realizada através de um repositório público no GitHub.

O histórico do Git fará parte da avaliação.

Grupos

Todos os participantes deverão possuir participação registrada no histórico do projeto. Cada integrante deverá realizar commits utilizando sua própria conta do GitHub.

O professor poderá utilizar o histórico do repositório para verificar a participação dos integrantes.

14. Branches

O desenvolvimento deverá utilizar branches. Não façam todo o projeto diretamente em uma única branch.

Exemplo:

```
main
|
├─ feature/modelos
├─ feature/repositories
├─ feature/services
├─ feature/controllers
└─ feature/testes
```

Os nomes das branches podem ser diferentes. O importante é demonstrar que houve organização do desenvolvimento.

15. Commits

Os commits deverão representar etapas reais do desenvolvimento.

Evitem um único commit como:

```
final
```

ou:

```
projeto pronto
```

Exemplos de commits adequados:

```
feat: cria entidade Cliente
feat: implementa ClienteRepository
feat: adiciona regra de cadastro de cliente
feat: cria ClienteController
test: adiciona testes do ClienteService
docs: atualiza README
```

Não é obrigatório utilizar exatamente esse padrão de mensagens. O importante é que o histórico seja compreensível.

16. Pull Requests

Não é obrigatório utilizar Pull Requests para todas as alterações.

Entretanto, grupos que utilizarem Pull Requests poderão registrá-los no histórico do projeto como evidência adicional de organização do trabalho.

17. README.md: obrigatório

O README deverá ser completo. Não será suficiente escrever apenas "Projeto desenvolvido para a disciplina."

O README deverá conter, no mínimo:

17.1 Identificação

- nome do projeto;
- nome dos participantes;
- turma;
- disciplina.

17.2 Descrição

Explique:

- qual problema o sistema resolve;
- quem utilizaria o sistema;
- qual é o objetivo da aplicação.

17.3 Funcionalidades

Liste as funcionalidades implementadas.

17.4 Regras de negócio

Liste as principais regras implementadas.

17.5 Arquitetura

Explique:

```
Controller
  ↓
Service
  ↓
Repository
```

Explique brevemente a responsabilidade de cada camada.

17.6 Tecnologias

Exemplo:

```
Java 21  
Maven  
Spring Boot  
JUnit  
Git  
GitHub
```

17.7 Como executar

Explique passo a passo:

```
git clone ...  
cd ...  
mvn spring-boot:run
```

Adapte os comandos ao projeto.

17.8 Como executar os testes

Exemplo:

```
mvn test
```

17.9 Estrutura do projeto

Inclua a árvore de diretórios.

17.10 Histórico do desenvolvimento

Explique brevemente como o projeto foi desenvolvido durante a semana. Exemplo:

- modelagem;
- criação das entidades;
- repositories;
- services;
- controllers;
- testes;
- documentação.

17.11 Participação dos integrantes

Explique resumidamente o que cada integrante desenvolveu. Exemplo:

Integrante	Contribuições
Aluno 1	Model + Repository
Aluno 2	Services
Aluno 3	Controllers
Aluno 4	Testes
Aluno 5	Documentação

18. Uso de Inteligência Artificial

A utilização de ferramentas de IA é permitida. Utilizar IA não elimina a responsabilidade de compreender o código entregue.

Durante a apresentação, qualquer integrante poderá ser questionado sobre:

- classes desenvolvidas;
- regras de negócio;
- dependências;
- arquitetura;
- funcionamento do Service;
- funcionamento do Repository;
- testes;
- decisões tomadas durante o desenvolvimento.

Se uma ferramenta de IA tiver sido utilizada de forma relevante, o grupo deverá registrar isso no README, incluindo:

- ferramenta utilizada;
- finalidade;
- prompts;
- como o código gerado foi revisado/adaptado.

19. O que não será avaliado

Não serão considerados diferenciais necessários para esta avaliação:

- interface gráfica;
- design visual;
- banco de dados;
- deploy;

- autenticação;
- APIs externas;
- Docker;
- arquitetura excessivamente complexa.

20. Apresentação

A apresentação acontecerá no dia da entrega.

Cada indivíduo/grupo deverá realizar uma apresentação curta e objetiva.

Sugestão de estrutura:

1. Problema

Qual problema o sistema resolve?

2. Solução

O que foi desenvolvido?

3. Demonstração

Mostrar rapidamente a aplicação funcionando.

4. Arquitetura

Explicar:

```
Controller
  ↓
Service
  ↓
Repository
```

5. Testes

Mostrar pelo menos um teste e explicar o que ele verifica.

6. GitHub

Mostrar o repositório e o histórico de desenvolvimento.

21. Regra mais importante

O objetivo não é entregar o maior sistema: é demonstrar que vocês conseguem aplicar, de forma organizada, os conceitos de Back-End estudados na Unidade 1.

Uma aplicação pequena que vocês conseguem explicar, executar, testar e defender durante a apresentação vale mais, para esta avaliação, do que uma aplicação cheia de tecnologias que vocês não conseguem explicar.